Reviewer Pack — Cyclic Burnside Orbit Count Lower-Bounds AC⁰ Size for PARITY

Entry 80d12cf7a27e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 00:49:13 UTC

Cyclic Burnside Orbit Count Lower-Bounds AC⁰ Size for PARITY

Entry ID: `80d12cf7a27e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 00:49:13 UTC

1. Conjecture

Field A (mathematical branch): Burnside-ring / equivariant orbit counting of cyclic group actions on labeled circuit DAGs (a corner of combinatorial representation theory rarely applied to circuit complexity) **Field B** (complexity object): AC^0 circuit size for PARITY_n, with the lifting-style intuition that gadget symmetry constrains the lifted communication matrix

Statement:

> Let C be an AC^0 circuit over inputs x_0,…,x_{n-1} computing PARITY_n with n a prime ≥ 3, depth(C)=d and size(C)=s, and let σ be the cyclic shift σ(i)=i+1 mod n acting on each gate g of C by simultaneously relabeling every input wire x_i appearing in g's recursive expansion to x_{σ(i)}. Define ψ(C) as the number of ⟨σ⟩-orbits of gates of C under the equivalence g ~ h iff some σ^k maps the lex-min canonical (op-type, sorted-children) form of g to that of h. Then for every such C computing PARITY_n we conjecture $\psi(C) \geq (1/(4d)) \cdot \log_2(s)$, refuted by any single PARITY-computing AC^0 circuit whose ratio $4 \cdot d \cdot \psi(C)/\log_2(s)$ drops below 1.

Rationale (proposer's reasoning):

> Lifting theorems translate query lower bounds into communication lower bounds by composing the function with a symmetry-breaking gadget; dually, when the target function (PARITY) is itself shift-invariant, a small circuit must reuse few genuinely distinct subcomputations, so the Z_n-orbit count of its gates becomes a Burnside-style proxy for the work that random restrictions are usually invoked to expose. Because ψ is defined on the circuit DAG rather than on the truth table, the invariant sits outside the Razborov–Rudich natural-proofs barrier; because the construction is purely combinatorial (no polynomial extension of the Boolean ring) it does not algebrize, matching the LIFTING approach's safe-direction profile.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9216b43f4897dd13`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 8 primes × 3 depths × 3 PARITY constructions × 30 seeds (2160 PARITY circuits), compute r = 4·d·ψ(C)/log_2(s) for each. SUPPORT iff every PARITY circuit yields r ≥ 1 (min r ≥ 1.0); FALSIFIED iff any single PARITY circuit yields r < 1. Negative-control non-PARITY circuits are excluded from the bound.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `AC0 circuit lower bounds PARITY symmetry orbit` - `Burnside cyclic group action circuit complexity lifting` - `equivariant gate symmetry bounded depth PARITY lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1708.00951v2] Canonical Heights and Monomial Maps: On Effective Lower Bounds for Points with Dense Orbit - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2005.06373v2] Counting Schur Rings over Cyclic Groups of Semi-prime Order - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random
import math
import json
from collections import defaultdict

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True
```

```
def generate_primes(start, end):
    primes = []
    for num in range(start, end + 1):
        if is_prime(num):
            primes.append(num)
    return primes

def matrix_mult(A, B):
    result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_pow(mat, power):
    result = [[1 if i == j else 0 for j in range(len(mat))] for i in range(len(mat))]
    while power > 0:
        if power % 2 == 1:
            result = matrix_mult(result, mat)
        mat = matrix_mult(mat, mat)
        power //= 2
    return result

def build_flat_dnf(n):
    gates = []
    for i in range(2 ** (n - 1)):
        term = []
        for j in range(n):
            if (i >> j) & 1:
                term.append(('NOT', j))
            else:
                term.append(('ID', j))
        gates.append(('AND', term))
    for _ in range(n - 1):
        new_gates = []
        for i in range(0, len(gates), 2):
            if i + 1 < len(gates):
                new_gates.append(('OR', [gates[i], gates[i + 1]]))
            else:
                new_gates.append(gates[i])
        gates = new_gates
    return gates[0]

def build_balanced_xor_tree(n):
    gates = [('ID', i) for i in range(n)]
    while len(gates) > 1:
        new_gates = []
        for i in range(0, len(gates), 2):
            if i + 1 < len(gates):
                new_gates.append(('XOR', [gates[i], gates[i + 1]]))
            else:
                new_gates.append(gates[i])
        gates = new_gates
    return gates[0]
```

```
def build_hastad_block_parity(n, block_size):
    gates = []
    for i in range(0, n, block_size):
        block = [('ID', j) for j in range(i, min(i + block_size, n))]
        if len(block) > 1:
            block_gate = ('XOR', block)
            gates.append(block_gate)
        else:
            gates.append(block[0])
    while len(gates) > 1:
        new_gates = []
        for i in range(0, len(gates), 2):
            if i + 1 < len(gates):
                new_gates.append(('XOR', [gates[i], gates[i + 1]]))
            else:
                new_gates.append(gates[i])
        gates = new_gates
    return gates[0]

def build_random_non_parity(n, depth):
    gates = [('ID', i) for i in range(n)]
    for _ in range(depth):
        new_gates = []
        for _ in range(len(gates)):
            op = random.choice(['AND', 'OR', 'XOR', 'NOT'])
            if op == 'NOT':
                gate = (op, [random.choice(gates)])
            else:
                gate = (op, random.sample(gates, 2))
            new_gates.append(gate)
        gates = new_gates
    return random.choice(gates)

def compute_psi(circuit, n):
    gate_hashes = {}
    canonical_forms = {}

    def hash_gate(gate):
        if isinstance(gate, tuple):
            op, children = gate
            if op == 'ID':
                return hash((op, children))
            child_hashes = sorted([hash_gate(child) for child in children])
            return hash((op, tuple(child_hashes)))
        else:
            return hash(gate)

    def get_canonical_form(gate, shift):
        if isinstance(gate, tuple):
            op, children = gate
            if op == 'ID':
                return (op, (shift + children) % n)
            child_forms = sorted([get_canonical_form(child, shift) for child in children])
            return (op, tuple(child_forms))
        else:
```

```
            return (('ID', (shift + gate) % n))

    for gate in circuit:
        gate_hashes[gate] = hash_gate(gate)

    for gate in circuit:
        min_form = None
        for shift in range(n):
            form = get_canonical_form(gate, shift)
            if min_form is None or form < min_form:
                min_form = form
        key = tuple(min_form)
        if key not in canonical_forms:
            canonical_forms[key] = []
        canonical_forms[key].append(gate)

    psi = len(canonical_forms)
    return psi

def compute_size(circuit):
    size = 0
    for gate in circuit:
        if isinstance(gate, tuple):
            size += 1 + compute_size(gate[1])
        else:
            size += 1
    return size

def run_trial(seed):
    random.seed(seed)
    n = random.choice([3, 5, 7, 11, 13, 17, 19, 23])
    depth = random.choice([2, 3, 4])
    construction = random.choice(['flat_dnf', 'balanced_xor', 'hastad', 'random_non_parity'])

    if construction == 'flat_dnf':
        circuit = build_flat_dnf(n)
    elif construction == 'balanced_xor':
        circuit = build_balanced_xor_tree(n)
    elif construction == 'hastad':
        block_size = random.randint(2, n // 2)
        circuit = build_hastad_block_parity(n, block_size)
    else:
        circuit = build_random_non_parity(n, depth)

    psi = compute_psi(circuit, n)
    s = compute_size(circuit)
    d = depth

    if s == 0:
        return {
            "metric_name": "psi_ratio",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
```

```
    r = (4 * d * psi) / math.log2(s)

    if construction == 'random_non_parity':
        return {
            "metric_name": "psi_ratio",
            "metric_value": r,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

    if r < 1:
        return {
            "metric_name": "psi_ratio",
            "metric_value": r,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, depth={d}, construction={construction}, r={r}"
        }
    else:
        return {
            "metric_name": "psi_ratio",
            "metric_value": r,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else generate_primes(100, 130)
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    holds = [r["conjecture_holds"] for r in results]

    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_trials")
    else:
        mean = sum(metric_values) / len(metric_values)
        std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
        support_fraction = sum(holds) / len(holds)

        if all(holds):
            print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
        else:
            counterexamples = [r["counterexample"] for r in results if not r["conjecture_holds"]]
            first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={firs
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c17bcfb6.py", line 225, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c17bcfb6.py", line 180, in run_trial
    psi = compute_psi(circuit, n)
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c17bcfb6.py", line 139, in compute_psi
    gate_hashes[gate] = hash_gate(gate)
                        ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c17bcfb6.py", line 126, in hash_gate
    return hash(gate)
           ^^^^^^^^^^
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError (unhashable list) in hash_gate before any trial completed, so no r values were produced and the pre-registered support condition could not be evaluated. | next: Fix the canonicalization in hash_gate by converting children lists to tuples (or using a frozen canonical form) before hashing, then re-run the full 2160-circuit grid.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	126113
2	preregistration	claude_max	opus	0	0	6115
3	novelty	claude_max	opus	0	0	3472
4	novelty	claude_max	opus	0	0	6684
5	test_gen	mistral	codestral-latest	0	0	314821
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	274196
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	184928
8	test_gen	mistral	codestral-latest	0	0	312953
9	judge	claude_max	opus	0	0	6960

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1236242 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 2, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/80d12cf7a27e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/80d12cf7a27e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/80d12cf7a27e.tar.gz` (if generated)